

Gestión Bases de Datos

MEMORIA TÉCNICA

David Roder

Índice

1. Tabla pacientes	3
Problemas detectados	3
Solución columna nif	4
Solución columna nombre_completo	4
Solución columna email	5
Solución columna f_nacimiento	5
Solución para los NULL	6
Resultado tras el arreglo	6
2. Tabla visitas	7
Problemas detectados	7
Solución columna fecha_visita	8
Solución para los NULL	8
Resultado tras el arreglo	9

1. Tabla pacientes

Problemas detectados

Tras la realización del escenario 1 de la práctica, al realizar un **SELECT** y un **DESCRIBE** de la tabla **pacientes**, obtendremos esta información.

id	nif	nombre_completo	email	tel_contacto	f_nacimiento
1	12345678A	Juan Carlos Ibañez	jc.ibanez@mail.com	600111222	12/05/1980
2	87654321B	Maria Lopez Sanz	m.lopez@gmail,com	611222333	1992.08.15
3	11223344C	LUIS MARTINEZ	luis@test@es	622444555	01-01-1975
5	22334455D	Ana Ruiz Pardo	NULL	NULL	20/10/2005
6	99999999Z	Paciente de Borrado	test@delete.com	123	2020-01-01
7	44556677X	Sonia Valverde	sonia.v@outlook.com	655999000	15/03/1988
10	55667788Y	Roberto Gomez	NULL	611000999	18/03/2026
11	99887766K	Elena Nito	NULL	NULL	2026.03.19

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
nif	char(9)	NO	UNI	NULL	
nombre_completo	varchar(200)	YES		NULL	
email	varchar(150)	YES		NULL	
tel_contacto	varchar(50)	YES		NULL	
f_nacimiento	varchar(50)	YES		NULL	

Como se aprecia tras el **SELECT** y el **DESCRIBE**, hay varios problemas que suceden en las columnas de la tabla **pacientes**.

1. Problemas en la columna **nif**: A pesar de estar puesto el tipo de dato **CHAR(9)**, que hace que el dato insertado debe de tener exactamente 9 caracteres, no hay una restricción de formato que hace la inserción de datos más completa y sencilla, donde los 8 primeros caracteres deben de ser números del 1 al 9 y el último carácter debe de ser una letra del abecedario excepto la I, Ñ, O, U.
2. Problema en la columna **nombre_completo**: Hay problemas con el formato de los nombres debido a que hay espacios sobrantes al principio de algún nombre y entre nombre y apellido. Además de diferentes usos de las mayúsculas en los nombres.
3. Problema en la columna **email**: Hay problemas con el formato de los email debido a emails que tienen duplicados los '@' y a emails que tienen mal escrito el dominio. Además no hay una restricción para que se cumpla un formato de una dirección de correo.

4. Problema en la columna **f_nacimiento**: Hay problemas debido a que las fechas no están estandarizadas y el tipo de dato no es **DATE**, lo que dará problemas en próximos **SELECT**.
5. Problema con los **NULL**: Para hacer la base de datos **gha_analytics** más compacta, alguna de las columnas que permiten obtener información (**email** o **tel_contacto**) para contactar con los clientes debería de ser obligatoria.

Solución columna **nif**

Para solucionar el problema con el formato de la columna **nif**, se usará un **CHECK** donde se verifique el formato de nif con **REGEXP**, este comando busca si el texto tiene una estructura determinada.

```
ALTER TABLE pacientes ADD CONSTRAINT chk_pacientes_nif CHECK (nif REGEXP  
'^[0-9]{8}[ABCDEFGHJKLMNPQRSTUVWXYZ]{1}$');
```

Solución columna **nombre_completo**

Para solucionar el problema con el formato de la columna **nif**, se harán dos **UPDATE**. En el primer **UPDATE** se usarán el comando **TRIM()**, que borrará los espacios finales al principio y/o al final del nombre que combinado con **REPLACE()** que reemplazará ' ' por ' '; en el segundo **UPDATE** se usará el comando **UPPER()** que pondrá el nombre en mayúsculas para que haya un formato común y no haya problemas en futuras consultas.

```
UPDATE pacientes SET nombre_completo = TRIM(REPLACE(nombre_completo, ' ', ' '));  
UPDATE pacientes SET nombre_completo = UPPER(nombre_completo);
```

Solución columna *email*

Para solucionar el problema con el formato de la columna **email**, se harán dos **UPDATE** y un **CHECK**. En el primer **UPDATE** se usarán dos **REPLACE()** combinados donde se reemplacen las ',' por '.' y luego en ese resultado se reemplacen los 'con' por 'com'. En el segundo **UPDATE** habrá un **WHERE** y se hará una combinación de los comandos **CONCAT()**, que combinará la partes de la dirección de correo, posteriormente se usará el comando **SUBSTRING_INDEX()**, que sustrae una parte de una palabra a partir de un carácter exacto, por último se usará **REPLACE()** para reemplazar ',' por '.'. En el **CHECK** se aplicará un **LIKE** donde se cumpla el formato de una dirección de correo.

```
UPDATE pacientes SET email = REPLACE(REPLACE(email, ',', '.'), 'con', 'com');

UPDATE pacientes SET email = CONCAT(SUBSTRING_INDEX(email, '@',
2), REPLACE(CONCAT('@', SUBSTRING_INDEX(email, '@', -1)), '@', '.')) WHERE email
LIKE ('%@%');

ALTER TABLE pacientes ADD CONSTRAINT chk_pacientes_email CHECK (email LIKE
' %@%.%');
```

Solución columna *f_nacimiento*

Para solucionar el problema con el formato de la columna **f_nacimiento**, se realizará un **UPDATE** con una estructura **CASE** y posteriormente un **ALTER TABLE**. En el **UPDATE**, se evaluarán cuatro patrones distintos mediante el comando **LIKE** para identificar los separadores (barras, puntos o guiones) y la posición del año; según el formato detectado, se aplicará la función **STR_TO_DATE()** utilizando los especificadores de formato de MySQL (%d, %m, %Y) para convertir los textos en valores de fecha estandarizados. Finalmente, se ejecutará un **ALTER TABLE** con la cláusula **MODIFY COLUMN** para transformar el tipo de dato de la columna de texto a **DATE**, asegurando así la integridad de la información y permitiendo el uso de funciones cronológicas.

```
UPDATE pacientes SET f_nacimiento = CASE
    WHEN f_nacimiento LIKE ('%//%') THEN STR_TO_DATE(f_nacimiento,
'%d/%m/%Y')
    WHEN f_nacimiento LIKE ('%.%.%) THEN STR_TO_DATE(f_nacimiento, '%Y.%m.%d')
    WHEN f_nacimiento LIKE ('__-__-__') THEN STR_TO_DATE(f_nacimiento,
'%d-%m-%Y')
    WHEN f_nacimiento LIKE ('____-__-__') THEN STR_TO_DATE(f_nacimiento,
'%Y-%m-%d')
END;

ALTER TABLE pacientes MODIFY COLUMN f_nacimiento DATE;
```

Solución para los NULL

Para solucionar los **NULL** con el objetivo de hacer una base de datos más compacta se elegirá la **tel_contacto** como obligatoria pues hay más registros de pacientes con teléfono que han tenido visitas que pacientes con email, además es una manera de comunicación más fiable. Se usará un **DELETE** para eliminar a los clientes que no tengan un teléfono o cuyo teléfono no cumpla con el formato del teléfono después de haber sido saneado en el escenario 1; después se añadirá la clave **NOT NULL** con **MODIFY COLUMN**.

```
DELETE FROM pacientes WHERE tel_contacto IS NULL OR LENGTH(tel_contacto) != 9;
ALTER TABLE pacientes MODIFY COLUMN tel_contacto CHAR(9) NOT NULL;
```

Resultado tras el arreglo

Tras hacer los cambios debido a los problemas concluidos si se hace un **SELECT** y **DESCRIBE** se podrán apreciar los cambios.

id	nif	nombre_completo	email	tel_contacto	f_nacimiento
1	12345678A	Juan Carlos Ibañez	jc.ibanez@mail.com	600111222	1980-05-12
2	87654321B	Maria Lopez Sanz	m.lopez@gmail.com	611222333	1992-08-15
3	11223344C	LUIS MARTINEZ	luis@test.es	622444555	1975-01-01
7	44556677X	Sonia Valverde	sonia.v@outlook.com	655999000	1988-03-15
10	55667788Y	Roberto Gomez	NULL	611000999	2026-03-18

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
nif	char(9)	NO	UNI	NULL	
nombre_completo	varchar(200)	YES		NULL	
email	varchar(150)	YES		NULL	
tel_contacto	char(9)	NO		NULL	
f_nacimiento	date	YES		NULL	

2. Tabla visitas

Problemas detectados

Tras la realización del escenario 1 de la práctica, al realizar un **SELECT** y un **DESCRIBE** de la tabla **visitas**, obtendremos esta información.

id	paciente_id	medico_id	fecha_visita	importe_sucio	descuento_aplicado	observaciones	copago_estimado
1	1	1	12/03/2026 10:30	150.50	10.00	Revisión anual	30.10
2	2	2	2026.03.13 09:00	80.00	NULL	NULL	16.00
3	3	1	14-03-2026 11:15	120.00	5.50	Dolor muscular	24.00
4	1	1	12/03/2026 10:30	150.50	10.00	NULL	30.10
8	1	2	18/03/2026 11:00	200.00	20.00	Urgencia	40.00

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
paciente_id	int	YES	MUL	NULL	
medico_id	int	YES	MUL	NULL	
fecha_visita	varchar(100)	YES		NULL	
importe_sucio	decimal(10,2)	YES		NULL	
descuento_aplicado	varchar(20)	YES		NULL	
observaciones	text	YES		NULL	
copago_estimado	decimal(10,2)	NO		NULL	

Como se aprecia tras el **SELECT** y el **DESCRIBE**, hay ciertos problemas que suceden en las columnas de la tabla **visitas**.

Problema en la columna **fecha_visita**: Hay problemas debido a que las fechas no están estandarizadas y el tipo de dato no es **DATETIME**, lo que dará problemas en próximos **SELECT**.

Problema con los **NULL**: Para hacer la base de datos **gha_analytics** más compacta, las columnas **paciente_id** y **medico_id** deben de ser obligatorias.

Solución columna *fecha_visita*

Para solucionar el problema con el formato de la columna *fecha_visita*, se ejecutará un **UPDATE** utilizando una condicional **CASE** para normalizar las entradas de texto antes de convertirlas. En este proceso, se empleará el comando **LIKE** para identificar los tres patrones de separadores presentes (barras, puntos y guiones) y determinar si el año se encuentra al inicio o al final de la cadena. Una vez detectado el patrón, se aplicará la función **STR_TO_DATE()** integrando tanto los especificadores de calendario (%d, %m, %Y) como los de tiempo (%H, %i), permitiendo así capturar horas y minutos con precisión. Finalmente, tras haber unificado el formato visual de los datos, la columna podrá ser alterada a un tipo **DATETIME** para garantizar la coherencia técnica y facilitar el análisis temporal de las visitas.

```
UPDATE visitas SET fecha_visita = CASE
    WHEN fecha_visita LIKE ('%/%/%') THEN STR_TO_DATE(fecha_visita, '%d/%m/%Y
%H:%i')
    WHEN fecha_visita LIKE ('%.%.%.') THEN STR_TO_DATE(fecha_visita, '%Y.%m.%d
%H:%i')
    WHEN fecha_visita LIKE ('__-__-____') THEN STR_TO_DATE(fecha_visita,
'%d-%m-%Y %H:%i')
END;

ALTER TABLE visitas MODIFY COLUMN fecha_visita DATETIME;
```

Solución para los NULL

Para solucionar los **NULL** con el objetivo de hacer una base de datos más compacta se añadirán la claves **NOT NULL** con **MODIFY COLUMN** a las columnas *paciente_id*, *medico_id*.

```
ALTER TABLE visitas MODIFY COLUMN paciente_id INT NOT NULL;

ALTER TABLE visitas MODIFY COLUMN medico_id INT NOT NULL;
```


Resultado tras el arreglo

Tras hacer los cambios debido a los problemas concluidos si se hace un **SELECT** y **DESCRIBE** se podrán apreciar los cambios.

id	paciente_ic	medico_ic	fecha_visita	importe_sucio	descuento_aplicado	observaciones	copago_estimado
1	1	1	2026-03-12 10:30:00	150.50	10.00	Revisión anual	30.10
2	2	2	2026-03-13 09:00:00	80.00	NULL	NULL	16.00
3	3	1	2026-03-14 11:15:00	120.00	5.50	Dolor muscular	24.00
4	1	1	2026-03-12 10:30:00	150.50	10.00	NULL	30.10
8	1	2	2026-03-18 11:00:00	200.00	20.00	Urgencia	40.00

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
paciente_id	int	NO	MUL	NULL	
medico_id	int	NO	MUL	NULL	
fecha_visita	datetime	YES		NULL	
importe_sucio	decimal(10,2)	YES		NULL	
descuento_aplicado	varchar(20)	YES		NULL	
observaciones	text	YES		NULL	
copago_estimado	decimal(10,2)	NO		NULL	